



Kmin

Academy of Computer Science

< mindful learning, meaningful mastery />

Specialized Structures

Some Notes from T-Bao

Computer Science Education Team

Kmin Academy of Computer Science

Specialized Structures

Edition: 1st Edition

Developed by: Computer Science Education Team

Published by: MinBook

Edition: 1st Edition

© 2026 Kmin Academy.

All rights reserved. This work is licensed for use by Kmin Academy students and partner institutions.

All rights reserved. No part of this publication may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of the publisher, except in the case of brief quotations embodied in critical reviews and certain other noncommercial uses permitted by copyright law.

About MinBook

MinBook is an open Computer Science textbook project by Kmin Academy, written and maintained by the CSE - Computer Science Education Team. Your feedback directly shapes the next revision of every book in the series – please reach out.

Feedback and Suggestions:

Mr. Bao Vu (T-Bao) – Project Lead, MinBook

✉ vdbao@kmin.edu.vn

Kmin Academy Contact Information

🌐 Website: kmin.edu.vn

📘 Facebook: fb.com/kminacademy

👥 AI & Software Learning Group: facebook.com/groups/kmin.ai.software

T-Bao Contact Information

📘 Facebook: facebook.com/tbaodevdi

🗨️ Learning Discord: discord.gg/jYKQ8B6mhb

Table of Contents

1	Why Your Toolbox Has a Gap	1
2	Tries: The Autocomplete Problem	2
3	Tries: Why Your Existing Tools Fall Short	2
4	Tries: Building the Intuition	3
5	Tries: What a Trie Actually Is	4
6	Tries: How It Works	5
7	Tries: A Worked Example	6
8	Tries: The Code	8
9	Tries: Cost, Compared Honestly	8
10	Common Mistakes with Tries	12
11	Best Practices for Tries	14
12	Remember Tries Like This	14
13	Union-Find: The Dynamic Connectivity Problem	15
14	Union-Find: Why Your Existing Tools Fall Short	15
15	Union-Find: The ADT Comes First	16
16	Union-Find: Building the Intuition	17
17	Union-Find: The Parent Array	18
18	Union-Find: The Naive Version, and Why It Breaks	18
19	Union-Find: Optimization One – Union by Rank	20
20	Union-Find: Optimization Two – Path Compression	21
21	Union-Find: The Combined Cost	23
22	Union-Find: A Worked Example	24
23	Union-Find: The Code	25
24	Union-Find: Finding the Redundant Connection	25
25	Common Mistakes with Union-Find	28
26	Best Practices for Union-Find	30
27	Remember Union-Find Like This	31
28	When Is a Specialized Structure Worth It?	31
29	Connections	32
30	Key Takeaway	34
31	Practice	34

🎯 Learning Objectives

- Explain why prefix queries defeat hash tables and why a trie solves them naturally
- Implement trie insert, search, and prefix operations, and analyze their cost in key length
- Explain the dynamic connectivity problem and describe the disjoint-set ADT
- Implement Union-Find with union by rank and path compression
- Evaluate when a purpose-built structure is worth its extra complexity

1 Why Your Toolbox Has a Gap

You have built an impressive toolbox. Arrays, linked lists, stacks, queues, hash tables, binary search trees, heaps – nine chapters of structures that store things and find them again.

Now try two ordinary problems with those tools.

Problem one. You are typing on your phone. After four letters, the keyboard offers you five complete words. Behind that suggestion sits a dictionary of three hundred thousand entries, and the keyboard must find every word starting with "prog" before your thumb reaches the next key.

Problem two. You are running a service that merges duplicate user accounts. All day long, two things happen in an unpredictable mix: “link these two accounts together,” and “are these two accounts already part of the same person?” There may be millions of accounts and millions of links.

Every structure you know handles these badly. Not catastrophically – badly enough to matter. In this chapter we will see why, and we will build two structures shaped for exactly these jobs.

The lesson underneath both halves of this chapter is one habit, worth stating before either structure appears: **look at the query pattern first, then choose the structure.** General-purpose structures answer “store it and find it” well. When a problem asks a narrower question over and

over, a structure built around that narrow question can beat them by a wide margin.

The two structures are the **trie**, which handles keys that are sequences of characters, and **Union-Find**, which handles grouping. Union-Find has a second reason to be here. In Chapter 19 you will meet Kruskal's algorithm, and Union-Find is the tool that makes it short. We are planting that seed now so the payoff feels like a reunion rather than a surprise.

2 Tries: The Autocomplete Problem

You are building the search box for an online store. A shopper types "lap". You need every product name that begins with those three letters, and you need it before they finish typing the fourth.

The catalogue has two million names, and it changes constantly. The question you are being asked is not "is "lap" a product name?" It is "*which keys begin with this?*" – a different question than any structure so far was designed to answer.

3 Tries: Why Your Existing Tools Fall Short

Let us take the obvious candidates seriously, because one of them is better than students usually expect.

A hash table (Chapter 8) is the wrong shape entirely. Hashing deliberately scrambles a key into a bucket index. That scrambling is the whole point – it spreads keys evenly. But it also destroys every relationship between keys. The strings "lap" and "laptop" land in unrelated buckets, so knowing where "lap" lives tells you nothing about where "laptop" lives. Answering a prefix query means examining every key, at a cost of $O(n \cdot L)$, where n is the number of stored keys and L is a key's length. For two million products, that is not a search box; it is a full scan.

A sorted array with binary search is genuinely decent. Sort the names alphabetically and every key sharing a prefix sits in one contiguous block, found in $O(L \log n)$. This is a real solution, and for a dictionary that never changes it may be the right one. Its weakness

is insertion: adding one product shifts everything after it, at $O(n)$. A catalogue that changes hourly makes that painful.

A balanced BST (Chapter 13) fixes insertion but not the underlying cost. Insertion and lookup both cost $O(L \log n)$: $\log n$ comparisons on the way down, and each comparison itself costs up to $O(L)$ because it compares whole strings. The key's length multiplies into every level of the search, not just the total step count.

Look at what all three have in common: each treats a string as one indivisible lump of data to be compared or hashed. And that is precisely the assumption a prefix query wants us to abandon.

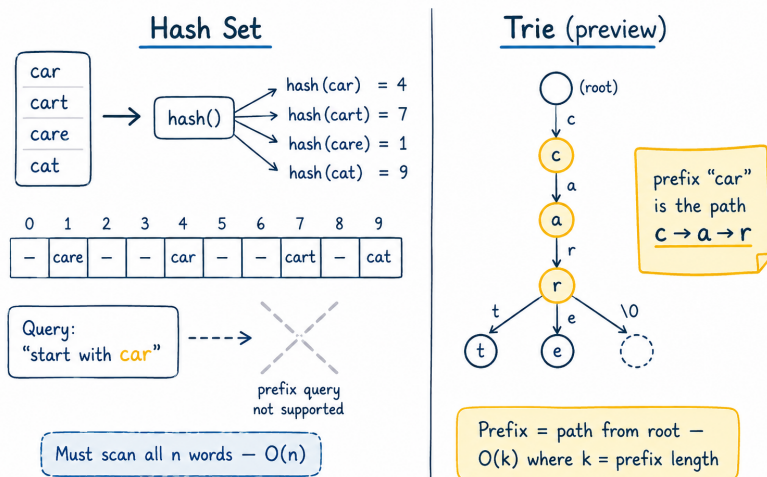


Figure 1. A hash set is blind to prefixes — hashing scrambles the very structure autocomplete needs.

4 Tries: Building the Intuition

Think about what "lap", "laptop", and "lapel" actually share. Not a hash value, not a sort position — they share their first three letters, physically, character for character. Every structure above stores those three letters three separate times and then works hard to rediscover the connection it just threw away.

So store them once. Picture a hallway. Every door along the hallway is labelled with a single letter. To spell a word, you walk: through the l

door, then the a door, then the p door. Words that begin alike walk the same hallway together and only split apart where they start to differ.

Now the prefix question answers itself. “Which words start with lap?” means “walk three doors, then look at everything reachable from where you are standing.” The structure did not have to search for the relationship. The relationship *is* the structure.

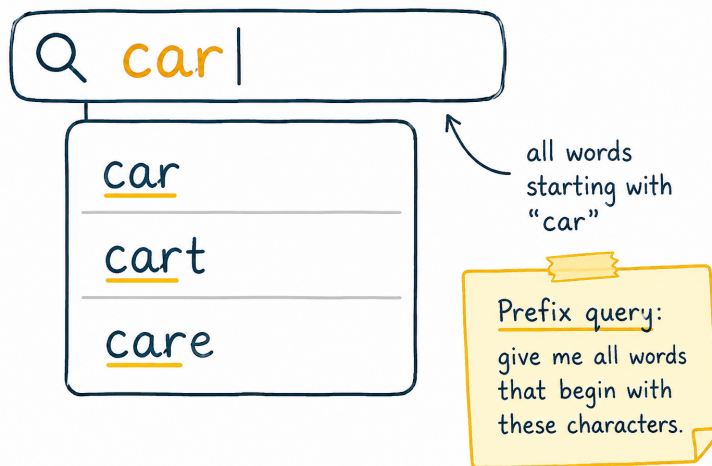


Figure 2. An autocomplete widget filters by prefix — a natural fit for a tree indexed by characters.

A trie is a branching road. Each turn you take is one letter, and the route you travelled spells the key. Two words share road for exactly as long as they share letters.

5 Tries: What a Trie Actually Is

Definition

trie

A **trie**, also called a *prefix tree*, is a rooted tree in which each edge is labelled with a single character. The path from the root to any node spells one prefix, and each node carries a flag marking whether that prefix is also a complete stored key.

Read that definition twice, because it contains the one genuine surprise of this chapter. **A trie node does not store a key.** It does not store a character either. The characters live on the edges, and the key lives in the path. A node's only job is to know where you can go next, and whether stopping here counts as a real word.

This breaks a habit built up over Chapters 13 and 14, where every node held a value you compared against. If you carry that habit into tries, nothing about them will make sense.

Definition

end-of-word marker

The **end-of-word marker** is a boolean flag on a node recording that the path ending at that node is a complete stored key, not merely a prefix of one.

The marker is not bookkeeping clutter. Without it, storing "car" and "cart" together is impossible: "car" would simply vanish inside the longer word, since its final node also happens to lie on the road to "cart".

6 Tries: How It Works

Every trie operation is the same motion – walk down from the root, one character at a time – and they differ only in what happens when the walk ends.

Insert. Start at the root. For each character of the key, look for an edge with that label. If it exists, follow it. If it does not, create it, then follow it. When the characters run out, set the end-of-word marker on the node you have reached. Inserting is just walking, building the road wherever it is missing.

Search. Start at the root and follow the characters. If an edge is missing at any point, the key is not stored, so report failure immediately. If you reach the end, you have found the *path* – now check the end-of-word marker. Path found and marker set means the key is stored. Path found and marker unset means you found a prefix of some other word, which is not the same thing.

Prefix check (starts_with). Identical to search, except you skip the marker check. Arriving at all is the answer.

Listing every key with a given prefix. Walk to the prefix node, then explore everything beneath it, collecting each node whose marker is set. Because no node stores its own key, you rebuild the key as you descend, appending each edge's character as you take it.

Notice what none of these operations do: none of them consult the number of stored keys. A trie holding five words and a trie holding five million words take exactly the same number of steps to look up a five-letter key.

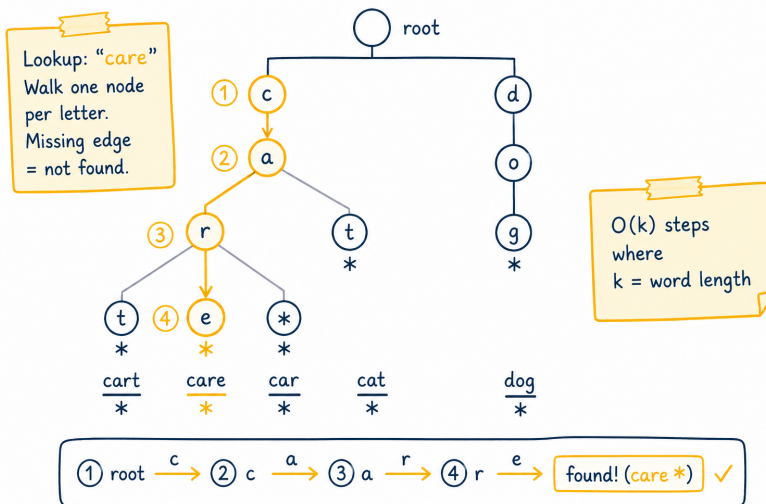


Figure 3. Lookup walks one node per letter; a missing edge means the prefix is absent.

7 Tries: A Worked Example

Example

Insert the five keys car, cart, care, cat, dog into an empty trie, in that order. Inserting car creates three nodes and marks the last one. Inserting cart reuses all three of those nodes and adds only one new node for t. Inserting care reuses c, a, r and branches off with e. Inserting cat reuses c, a and branches at the second character. Inserting dog shares nothing and builds a fresh branch from the root.

The finished structure appears in Figure 4.

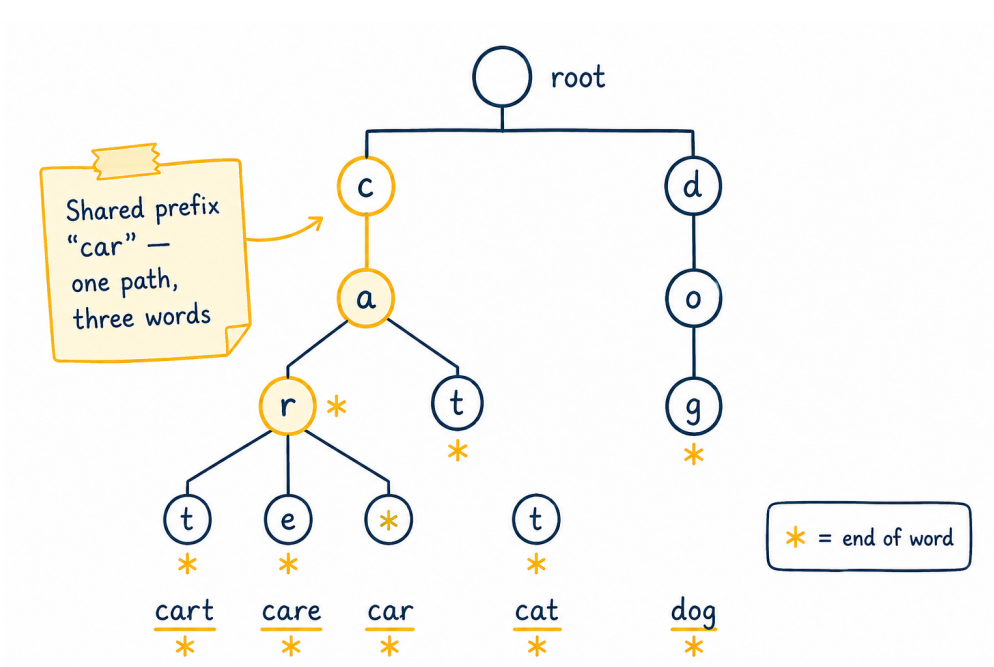


Figure 4. A trie storing car, cart, care, cat, and dog; asterisks mark complete words.

Three queries against Figure 4 show the mechanism at work.

search("car") walks c, a, r, arrives at a node whose marker is set, and returns true. search("ca") walks c, a, arrives successfully — and returns **false**, because the marker there is not set. "ca" is a hallway, not a destination. And search("cab") fails earlier still: after c and a, there is no edge labelled b, so the walk stops without examining any of the other four words.

That middle query is the one to remember. It is the sole reason the end-of-word marker exists, and it is the most common trie bug in existence.

One more trace pays off the opening hook directly. words_with_prefix("ca") first walks to the node reached by c, a — the same node search("ca") stopped at. From there it explores every branch beneath: down r, it finds r itself marked ("car"), then t marked ("cart") and e marked ("care"); down t from the a node, it finds t marked ("cat"). It never visits the d branch at all, because that branch is not reachable from the prefix node. The result is ["car", "cart", "care", "cat"] — the

walk to the prefix costs $O(L)$, and everything after that costs only as much as the output.

8 Tries: The Code

The node definition, shown in `lst. 0.1`, is worth pausing on for one specific reason: count its fields.

Listing 0.1. A trie node holds no key of its own — only outgoing edges and a marker.

```
1 class TrieNode:
2     # No key field, and no character field. The character that
  leads
3     # here is the map entry in the parent that points at this node.
4     children      = empty map from character to TrieNode
5     is_end_of_word = false
```

The three core operations follow in `lst. 0.2`. All three share a single helper, because all three do the same walk.

The autocomplete promise from the opening is delivered by `lst. 0.3`, which is only nine lines because the walk has already done the hard part.

9 Tries: Cost, Compared Honestly

Complexity Analysis

Dimensions: time, space

Let L be the length of the key being looked up, n the number of stored keys, and A the size of the alphabet.

- `insert`, `search`, `starts_with`: $O(L)$ time. Each step follows exactly one edge, and there are L steps. The count n never appears.
- `words_with_prefix`: $O(L)$ to reach the prefix, plus time proportional to the size of the answer.
- Space: $O(n \cdot L)$ nodes in the worst case, when no keys share prefixes. Each node also carries a container for up to A child links.

Listing 0.2. Trie insert, search, and prefix check, built on one shared walk.

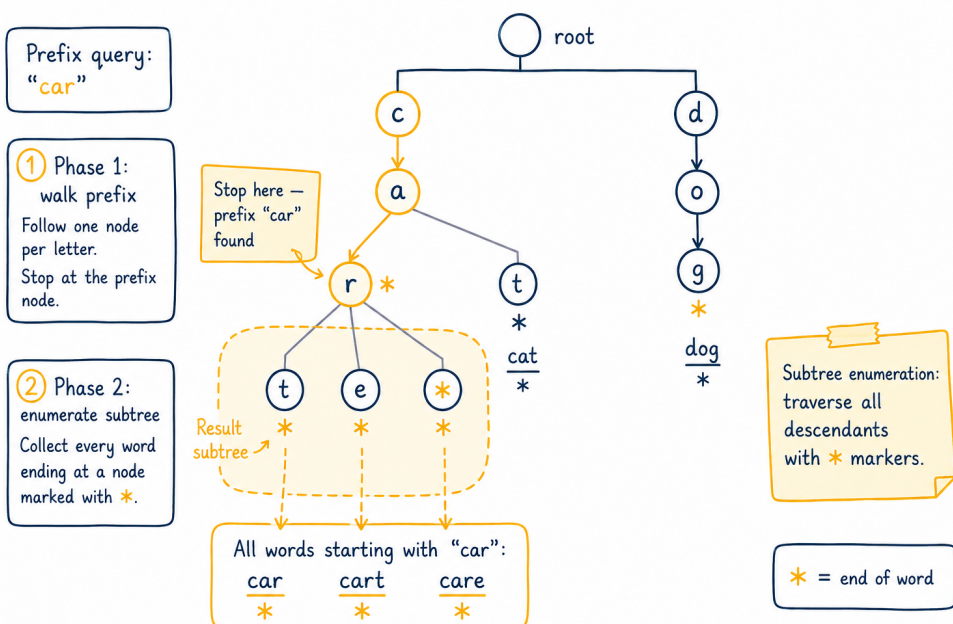
```
1 class Trie:
2     root = new TrieNode()
3
4     function insert(word):
5         node = root
6         for each ch in word:
7             # Build the road where it is missing, then drive on.
8             if ch not in node.children:
9                 node.children[ch] = new TrieNode()
10                node = node.children[ch]
11            # Without this line, "car" disappears inside "cart".
12            node.is_end_of_word = true
13
14        # Walks the path and returns where it ended, or null if it
15        broke.
16        function walk(prefix):
17            node = root
18            for each ch in prefix:
19                if ch not in node.children:
20                    return null
21                node = node.children[ch]
22            return node
23
24        function search(word):
25            node = walk(word)
26            # Arriving is not enough – the marker decides.
27            return node ≠ null and node.is_end_of_word
28
29        function starts_with(prefix):
30            # Here, arriving IS enough.
31            return walk(prefix) ≠ null
```

Listing 0.3. Collecting every stored key beneath a given prefix.

```

1  function words_with_prefix(prefix):
2      results = empty list
3      start   = walk(prefix)
4      if start == null:
5          return results
6      collect(start, prefix, results)
7      return results
8
9      # `so_far` rebuilds the key as we descend, because no node
10     stores it.
11     function collect(node, so_far, results):
12         if node.is_end_of_word:
13             results.append(so_far)
14         for each (ch, child) in node.children:
15             collect(child, so_far + ch, results)

```

**Figure 5.** A prefix query stops at the prefix node; its subtree enumerates every completion.

That space line is where honesty matters. A trie’s speed comes from spending memory, and if your keys share few prefixes you pay the full price and gain almost nothing. A hash table storing the same keys is usually smaller and, for pure exact-match lookup, usually faster in practice, because one hash computation beats L pointer hops through scattered memory.

 Concept Comparison

Compares to: trie versus hash table versus BST for string keys

Table 1. When each structure wins for string keys.

Operation	Hash table	Balanced BST	Trie
Exact lookup	$O(L)$ average	$O(L \log n)$	$O(L)$ worst case
Does any key start with P?	$O(n \cdot L)$	$O(L \log n)$	$O(L)$
List all keys starting with P	$O(n \cdot L)$	$O(L \log n) +$ output	$O(L) +$ output
Keys come out sorted?	No	Yes	Yes
Memory	Smallest	Moderate	Largest

The decision rule hiding in Table 1 is short. **If you only ever ask “is this exact key present?”, use a hash table.** If you ask “which keys begin with this?”, or you need keys in alphabetical order as a side effect, a trie earns its memory.

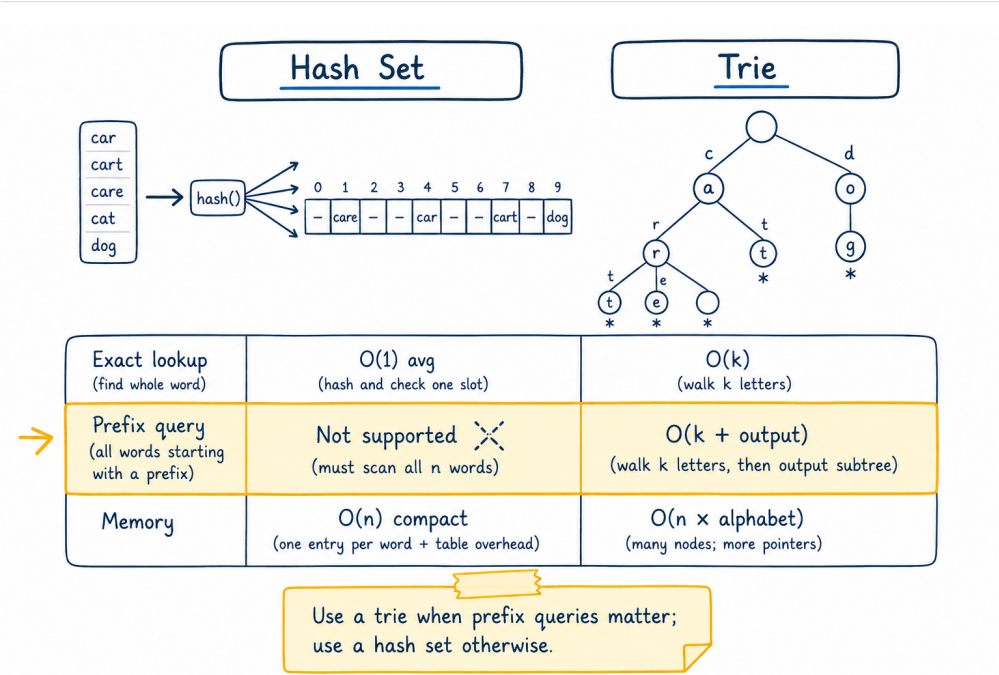
 Real-World Application

System: predictive-text keyboards and IP routing tables

Smartphone keyboards such as Gboard use trie-based dictionaries to offer completions after every keystroke. Routers use the same idea, matching a destination address against a table of address prefixes to decide where to forward each packet.

 Note

There is a well-known space optimization in which any chain of single-child nodes is collapsed into one node holding a whole substring, giving



The second mistake is the one that survives code review and fails in production.

⊗ Common Misconception

Belief – If the path for a string exists in the trie, the string is a stored key.

Correction – Path existence proves only that the string is a prefix of some stored key; the end-of-word marker decides membership.

Reason – A successful walk feels like a successful search, so the marker check looks optional. Storing car and cart together makes the difference visible: the path for car exists either way, but only one situation means car was actually inserted.

The remaining two come from over-enthusiasm for a new tool rather than from a coding slip.

⊗ Common Misconception

Belief – A trie is always the better choice for string keys.

Correction – For exact-match lookup with no prefix requirement, a hash table is normally faster in practice and much smaller in memory.

Reason – The trie's advantage is entirely about prefixes. Remove that requirement and you are paying a large memory cost, plus L scattered pointer dereferences, for a query a single hash computation would have answered.

⊗ Common Misconception

Belief – Trie operations get slower as the trie grows.

Correction – Lookup cost is $O(L)$ in the length of the key and is independent of how many keys are stored.

Reason – Nine chapters of using n as the input size makes n feel like the answer to every complexity question. The right first question is always: what is n here? For a trie, the quantity that varies is how far you walk, not how much is stored.

11 Best Practices for Tries

We recommend a small number of habits that keep trie code correct and affordable.

Choose the child container deliberately. A fixed array of size A is fastest but wastes memory when the alphabet is large or the trie is sparse; a hash map costs a little more per step and only stores edges that exist. For lowercase English, either is fine; for Unicode, use a map.

Normalize keys before insertion – lowercasing, trimming, stripping accents – and normalize queries the same way. A trie compares characters exactly, so "Laptop" and "laptop" become two unrelated branches unless you prevent it.

Cap the results of a prefix query. `words_with_prefix("a")` on a large dictionary may return most of the dictionary, so real autocomplete stops the collection walk once it has enough results.

Finally, measure before adopting. If your keys are long and share almost nothing, a trie may cost several times the memory of a hash table for a benefit you never use.

12 Remember Tries Like This

Carry this one image forward, because it is the whole chapter in miniature.

Memory Anchor

A trie is a **hallway of lettered doors**. To spell a word you walk through it door by door, and words that begin alike share hallway until the moment they differ. Some doors have a **doormat** – that is the end-of-word marker. Reaching a door means the prefix exists. Reaching a *doormat* means the word exists.

With that picture fixed, the formal rules from this section – path as key, marker as membership, cost measured in L rather than n – should all feel like restatements of something you already know.

Key Takeaway

- A trie stores characters on its edges; the path from the root spells the key, and nodes hold no key at all
- Lookup costs $O(L)$ in the key's length, independent of how many keys are stored
- The end-of-word marker is what separates “this is a prefix” from “this is a stored word”
- Choose a trie for prefix queries and sorted output; choose a hash table for pure exact-match lookup

13 Union-Find: The Dynamic Connectivity Problem

Now to the second half, and a very different kind of question.

Picture a social platform that is still growing. All day, two kinds of events arrive interleaved and in unpredictable order:

- **Link:** `connect(alice, bob)` – these two people now know each other.
- **Query:** `same_group(alice, dana)` – are these two people reachable from one another through the links added so far?

There may be ten million people and a hundred million events. Note carefully what the query asks and what it does *not* ask. It does not ask *how* Alice reaches Dana, or *who else* is in their group, or how large the group is. It asks one yes-or-no question: same group or not?

This is the **dynamic connectivity problem** – dynamic because the links keep arriving, and you must answer queries correctly at every moment, not just at the end.

14 Union-Find: Why Your Existing Tools Fall Short

Students reliably propose two solutions, and both are instructive.

Idea one: search whenever asked. Store the links, and when a query arrives, start at Alice and explore outward until you either find Dana or run out of people. This is correct. It is also $O(n)$ per query, repeated a hundred million times, and it redoes work it has already done thousands of times before.

Idea two: give every person a group label. Store an array `group[ ]`, so `same_group(a, b)` is a single comparison — beautifully fast. But now consider what `connect` must do when two different groups merge: every member of one group needs its label rewritten. One merge can cost $O(n)$, and merging repeatedly costs $O(n^2)$ overall.

Look at the pattern. Idea one makes queries expensive; idea two makes merges expensive. Each is fast at one operation and slow at the other, because each stores the wrong thing. What we want is a representation where **both** operations are cheap, and getting there requires giving up something we did not actually need.

15 Union-Find: The ADT Comes First

Before any representation, let us do what Chapter 4 taught and specify the interface.

Definition

disjoint-set ADT

The **disjoint-set ADT**, universally known as **Union-Find**, maintains a collection of elements partitioned into non-overlapping groups, and supports three operations:

- `make_set(x)` — place x alone in a group of its own
- `find(x)` — return an identifier for the group containing x
- `union(a, b)` — merge the group containing a with the group containing b

`connected(a, b)` is a convenience built from `find`: two elements are in the same group exactly when their group identifiers match.

Read the list again and notice the operation that is missing. There is no “list the members of my group,” and no “undo the last union.” Union-Find answers “*same group?*” and essentially nothing else.

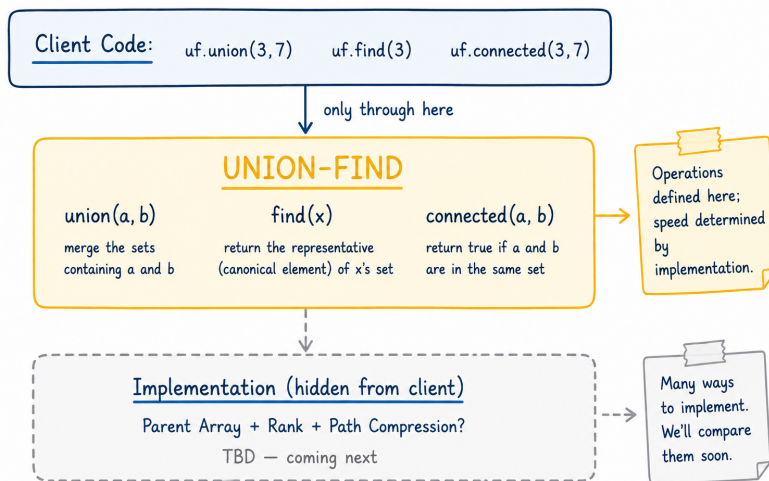


Figure 7. Union-Find promises three operations; the speed of those operations is a separate design choice.

That restriction is not a limitation to apologize for. It is the source of the speed. Because we never promise to enumerate a group, we never have to store the group.

16 Union-Find: Building the Intuition

Every group elects a president. Nobody keeps a membership list; each person simply knows one thing — who they report to. Follow that chain of reporting upward for long enough and you reach the president, who reports to nobody but themselves.

Now both operations become easy. To ask “are Alice and Dana in the same club?”, walk up from each of them and compare presidents. Same president, same club. To merge two clubs, the two presidents meet and one of them agrees to report to the other. That is a single change. Nobody else is told, nobody’s records are rewritten, and yet from that instant every member of both clubs walks up to the same president.

That is the trick in full. One person changes; a million memberships change with them.

Union-Find is a room full of clubs. Each member points at one person above them, and the chain always ends at a president. “Same club?” is really “same president?”

17 Union-Find: The Parent Array

Turning clubs and presidents into code needs exactly one array.

Number the elements 0 to $n - 1$ and keep an array `parent`, where `parent[x]` is the element that x reports to. A president is represented by pointing at itself: `parent[x] = x` means x is a root. `find(x)` follows parent links upward until it reaches a self-pointing element. This is the second time in the book that a tree lives entirely inside a plain array with no node objects at all – Chapter 14’s heap did the same thing for a different reason.

The resulting picture is a collection of trees – one tree per group, with the group’s representative at the root. But be careful about what kind of tree this is.

Warning

These trees are **not** search trees. There is no ordering between a node and its children, no in-order traversal that means anything, and no lookup by key. The only fact a Union-Find tree encodes is “who do I report to.” Its shape carries no information at all – which is exactly why we will be free to change that shape whenever it suits us.

Note also that the arrows run the opposite way from every tree so far. A binary search tree points downward, from parent to children, because you search by descending. Union-Find points upward, from child to parent, because you always travel toward the root.

18 Union-Find: The Naive Version, and Why It Breaks

The straightforward implementation writes itself:

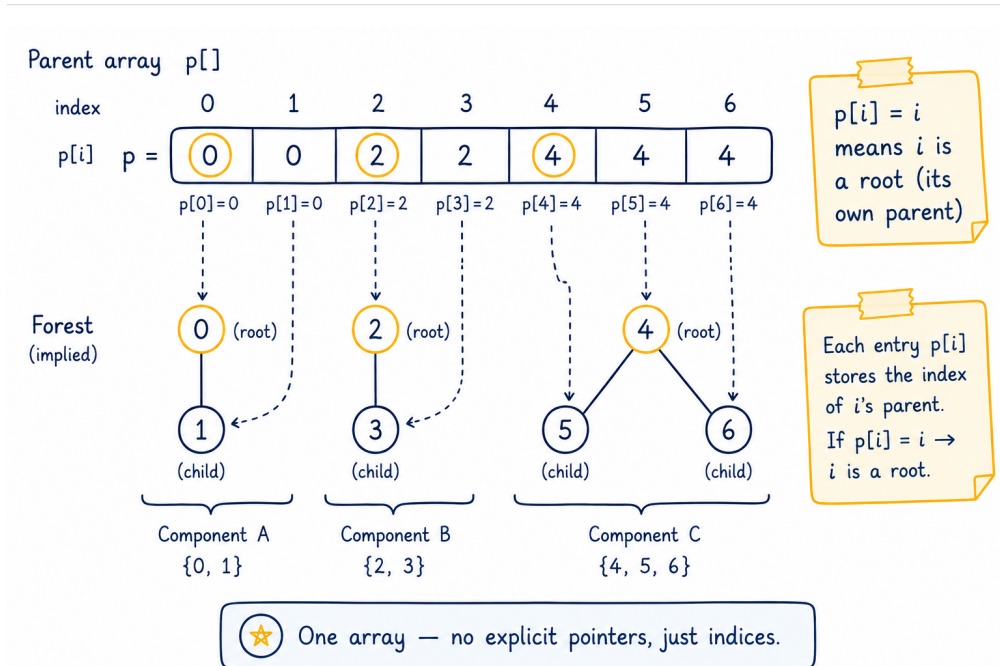


Figure 8. The parent array is the entire data structure — each entry stores the index of its parent.

- $\text{find}(x)$: follow parent links until $\text{parent}[x] = x$.
- $\text{union}(a, b)$: find both roots; if they differ, point one root at the other.

It is correct. It is also, in the worst case, no better than the ideas we rejected — and seeing exactly how it fails is what makes the rest of this section make sense.

Example

Start with seven elements, 0 through 6, each alone. Then apply this sequence:

$\text{union}(0,1)$, $\text{union}(1,2)$, $\text{union}(2,3)$, $\text{union}(3,4)$, $\text{union}(4,5)$, $\text{union}(5,6)$

Trace it with the naive rule “point a 's root at b 's root.” The first call sets $\text{parent}[0] = 1$. The second finds 0's root — now 1 — and points it at 2. The third points 2 at 3, and so on. The result is shown on the left of Figure 9.

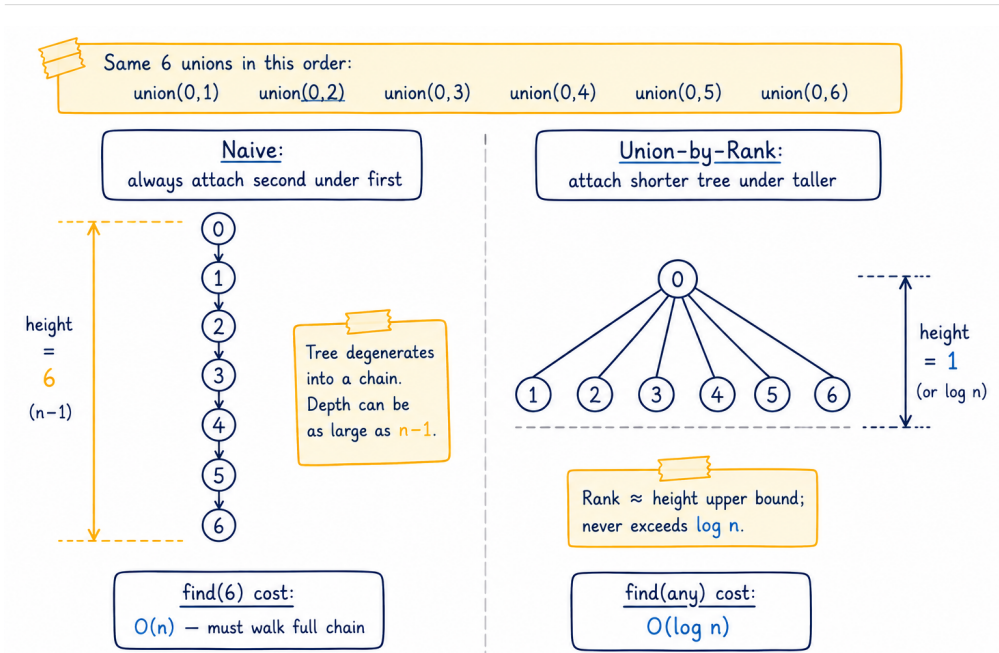


Figure 9. The same six unions under the naive rule and under union by rank.

The left side of Figure 9 is a linked list wearing a tree costume. `find(0)` costs $O(n)$, and so does every operation that touches the bottom of the chain.

You have seen this failure before. In Chapter 13, inserting sorted keys into a binary search tree produced exactly this shape and destroyed exactly this performance. The lesson repeats: **a tree that is allowed to grow tall is not a tree, it is a list**. So the fix must be the same in spirit — control the height.

19 Union-Find: Optimization One — Union by Rank

The naive union picks arbitrarily which root becomes subordinate. That arbitrary choice is what allows chains. Choosing deliberately prevents them.

 Definition**union by rank**

Union by rank keeps an integer $\text{rank}[x]$ for every root, an upper bound on the height of its tree. When merging, the root of lower rank is attached beneath the root of higher rank. If the two ranks are equal, either may go beneath the other, and the surviving root's rank increases by one.

The reasoning is worth one sentence. Hanging a short tree under a tall one does not change the tall tree's height, so the merged tree is no taller than it was. Height only grows when two equally tall trees combine, and that requires the group to roughly double in size – which cannot happen more than $\log_2 n$ times.

 Example

Replay the identical adversarial sequence with union by rank. The first call joins two rank-0 roots, so 1 is attached under 0 and $\text{rank}[0]$ becomes 1. Every later call joins a rank-1 root to a lonely rank-0 element, so the lonely element is attached under 0 and no rank changes.

The outcome is the right side of Figure 9: a flat star of height 1. The sequence that produced a six-hop chain now produces a one-hop lookup.

With union by rank, tree height is bounded by $\log_2 n$, so every find costs $O(\log n)$ in the worst case. That is already a complete, respectable solution. But we can do better, and the way we do it is genuinely surprising.

20 Union-Find: Optimization Two – Path Compression

Here is an observation that feels almost like cheating. Every time $\text{find}(x)$ runs, it walks from x all the way to the root – and along the way it *learns the root* for every node it passes. Then it throws that knowledge away and walks the same path again on the next query.

Definition

path compression

Path compression is the technique of re-pointing every node visited during a find directly at the root, so that later find calls on any of those nodes reach the root in a single step.

So find does two passes: **Pass 1** walks up to locate the root; **Pass 2** walks the same path again, setting each node's parent to that root.

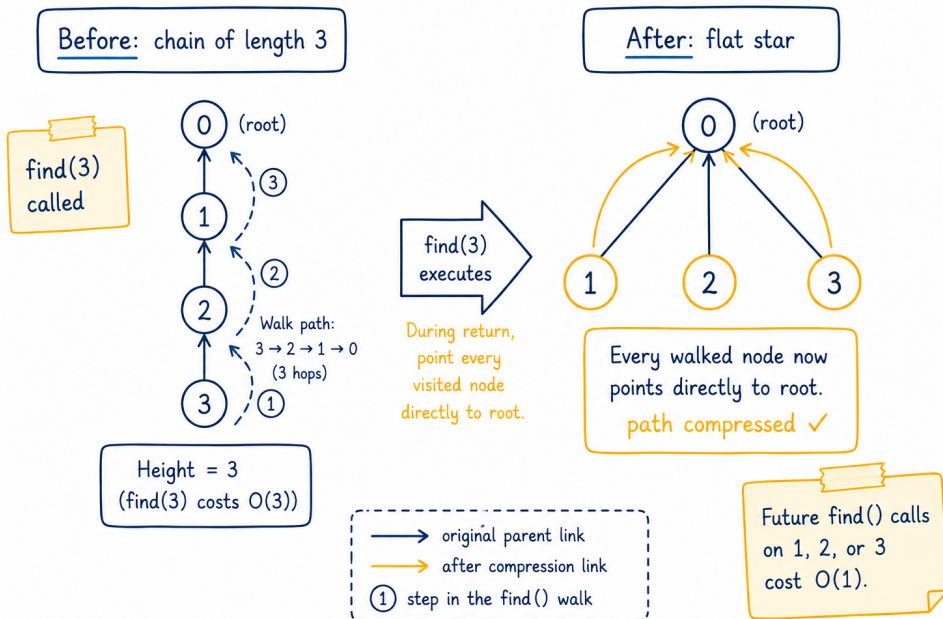


Figure 10. One call to find(0) flattens every node it walked past.

Two things about Figure 10 deserve attention. First, the *answer* did not change — find(0) returns 3 either way; compression is pure housekeeping performed during a read. Second, it repairs damage rather than preventing it: it flattens paths that have already grown, and only the paths you actually touch.

That is why the two optimizations are partners, not rivals. Union by rank is **prevention** — it stops tall trees from forming. Path compression is **repair** — it flattens whatever tall paths do get walked. Neither makes the other redundant, and a chapter-15 debugging exercise will ask you to show exactly why one alone is not enough.

21 Union-Find: The Combined Cost

Concept Comparison

Compares to: four Union-Find variants

Table 2. The four Union-Find variants and their per-operation cost.

Version	Cost per operation	Why
Naive	$O(n)$ worst case	Trees may degenerate into a chain
Union by rank only	$O(\log n)$ worst case	Height is bounded by $\log_2 n$
Path compression only	$O(\log n)$ amortized	Walked paths flatten over time
Both together	$O(\alpha(n))$ amortized	Effectively constant for any real input

The final row of Table 2 needs a plain-language gloss. The symbol $\alpha(n)$ is the *inverse Ackermann function*, and the only property you need is this: it grows so slowly that for any n a computer could ever store – every atom in the observable universe, and far beyond – $\alpha(n)$ is at most 5.

Complexity Analysis

Dimensions: time, space

With both optimizations, for n elements and m operations:

- `make_set`: $O(1)$
- `find`, `union`, `connected`: $O(\alpha(n))$ amortized – at most about 5 steps in practice
- Total for m operations: $O(m \cdot \alpha(n))$
- Space: $O(n)$ – two integer arrays, `parent` and `rank`, and nothing else

That space line deserves a moment. A structure tracking the grouping of ten million elements needs two integer arrays. There are no nodes, no pointers, and no allocation during operation.

22 Union-Find: A Worked Example

Example

Seven elements, 0 through 6, with both optimizations enabled. Ranks all start at 0. Table 3 shows the parent array after each operation.

Table 3. A full Union-Find trace with union by rank and path compression.

#	Operation	parent after	What happened
0	<i>(initial)</i>	[0,1,2,3,4,5,6]	Every element is its own root
1	union(0,1)	[0,0,2,3,4,5,6]	Equal ranks; 1 joins 0, rank[0] 1
2	union(2,3)	[0,0,2,2,4,5,6]	Equal ranks; 3 joins 2, rank[2] 1
3	union(1,3)	[0,0,0,2,4,5,6]	Roots 0 and 2, both rank 1; 2 joins 0, rank[0] 2
4	connected(0,3)	[0,0,0,0,4,5,6]	True. find(3) walked 3→2→0 and compressed 3
5	union(4,5)	[0,0,0,0,4,4,6]	Equal ranks; 5 joins 4, rank[4] 1
6	connected(0,5)	unchanged	False — roots 0 and 4 differ
7	union(5,6)	[0,0,0,0,4,4,4]	rank[4]=1 beats rank[6]=0; 6 joins 4
8	union(3,5)	[0,0,0,0,0,4,4]	rank[0]=2 beats rank[4]=1; 4 joins 0
9	connected(0,6)	[0,0,0,0,0,4,0]	True. find(6) walked 6→4→0 and compressed 6

Two rows in Table 3 carry the lesson. Row 4 is a *query* that modified the array — that is path compression, and it is not a mistake. Row 8 is a merge of two multi-element groups that changed exactly one entry, because presidents merge, not members.

23 Union-Find: The Code

The single most important line in lst. 0.4 is `parent[root_b] = root_a`. It operates on *roots*, obtained from `find`, never on the arguments `a` and `b` directly. Writing `parent[a] = b` instead produces code that passes small tests and silently loses groups on larger ones.

💡 Tip

`union` returning a boolean – true if a real merge happened, false if the two were already together – costs nothing and turns out to be exactly the signal several algorithms need. We use it immediately in the application that follows.

24 Union-Find: Finding the Redundant Connection

Here is a small problem that shows the structure earning its keep.

You are given a list of connections between n items, added one at a time. Some connection in that list is redundant: by the time it arrives, its two endpoints are *already* linked through earlier connections, so it adds nothing new. Find the first such connection.

With Union-Find, the entire solution is the return value of `union`:

The whole algorithm is five lines and runs in $O(m \cdot \alpha(n))$ for m connections. Solving it with repeated searching would take $O(m \cdot n)$.

🌐 Real-World Application

System: account merging and image segmentation

Systems that de-duplicate user records use Union-Find to check “is this already the same person?” without materializing the merged record. The same structure labels connected regions in image processing: each pixel is unioned with matching neighbours, and the roots become region identifiers.

Listing 0.4. Union-Find with union by rank and path compression.

```

1 class UnionFind:
2     parent = array of size n
3     rank   = array of size n
4
5     function make_sets(n):
6         for i from 0 to n - 1:
7             parent[i] = i    # self-pointing means "I am a root"
8             rank[i]   = 0
9
10    function find(x):
11        # Pass 1: climb to the root.
12        root = x
13        while parent[root] ≠ root:
14            root = parent[root]
15        # Pass 2: path compression. We already know the root for
16        # every node we just passed, so record it instead of
17        # discarding it and re-walking next time.
18        while parent[x] ≠ root:
19            next_node = parent[x]
20            parent[x] = root
21            x = next_node
22        return root
23
24    function union(a, b):
25        root_a = find(a)
26        root_b = find(b)
27        if root_a = root_b:
28            return false          # already together; nothing
merged
29        # Union by rank: hang the shallower tree under the deeper
one,
30        # so the merged tree is no taller than it already was.
31        if rank[root_a] < rank[root_b]:
32            parent[root_a] = root_b
33        else if rank[root_a] > rank[root_b]:
34            parent[root_b] = root_a
35        else:
36            parent[root_b] = root_a
37            rank[root_a] = rank[root_a] + 1
38        return true
39
40    function connected(a, b):
41        return find(a) = find(b)

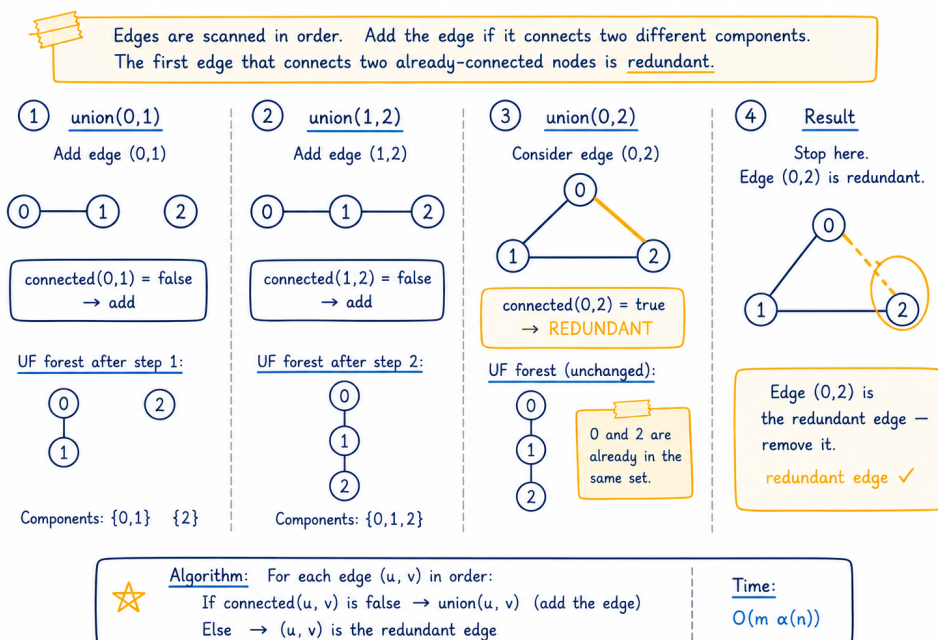
```

Listing 0.5. Detecting the first connection that adds nothing new.

```

1 function first_redundant(n, connections):
2     uf = new UnionFind()
3     uf.make_sets(n)
4     for each (a, b) in connections:
5         # union returns false exactly when a and b were already
linked,
6         # which means this connection closes a loop.
7         if uf.union(a, b) == false:
8             return (a, b)
9     return none

```

**Figure 11.** Scanning edges in order, the first one to join two already-connected nodes is the redundant edge.

■ Note

Keep Ist. 0.5 in mind. In Chapter 19, Kruskal's algorithm builds a minimum spanning tree by considering edges in increasing weight order and keeping each one that joins two previously separate groups. That test is exactly the union call written here.

25 Common Mistakes with Union-Find

Six beliefs cause most Union-Find trouble. The first is a coding slip; the rest are about what the structure actually promises.

⊗ Common Misconception

Belief – `union(a, b)` should set `parent[a] = b`.

Correction – union must operate on the two roots returned by find, never on the arguments directly.

Reason – Pointing a non-root at something detaches it from its own subtree, which silently splits a group in two. On small tests where most elements are still roots, the buggy version produces correct answers, so the error survives until the input gets big.

The next error is subtler, because both optimizations look good in isolation on small traces.

⊗ Common Misconception

Belief – Union by rank and path compression do the same job, so one of them is enough.

Correction – Rank prevents tall trees from forming; compression flattens tall paths that already exist. Each alone gives $O(\log n)$; only together do they give near-constant cost.

Reason – On a small hand-traced example both produce flat-looking trees, which makes them appear interchangeable. They act at different moments – rank during a merge, compression during a lookup – so neither can do the other's work.

This one comes from carrying an assumption over from Chapter 13.

⊗ Common Misconception

Belief – Union-Find trees are search trees, like a BST.

Correction – A Union-Find tree encodes nothing but ‘who do I report to.’ There is no ordering between a node and its children and no meaningful traversal by key.

Reason – The diagrams look similar to a BST, and Chapter 13 conditioned you to expect meaning in a tree’s shape. Here, shape is disposable – that is precisely why path compression is allowed to rearrange it freely.

This one concerns the near-constant cost claim itself.

⊗ Common Misconception

Belief – (n) is just a fancy way of writing $O(1)$.

Correction – (n) is not literally constant – it does grow, just extraordinarily slowly. Saying ‘effectively constant’ describes its behavior on any input a computer could hold, not a claim that the function is flat.

Reason – The chapter itself uses the phrase ‘effectively constant,’ and it is easy to round that down to a formal $O(1)$. Proving the (n) bound is genuinely difficult and is beyond this book’s scope; the honest statement is ‘at most about 5 steps in practice,’ not ‘always 1 step.’

Two more misunderstandings concern what the structure promises rather than how it is coded.

⊗ Common Misconception

Belief – Union-Find stores which elements are in each group.

Correction – It stores only a parent pointer per element; it can tell you whether two elements share a group, but not who else is in one.

Reason – The word set in disjoint-set suggests an enumerable collection. Union-Find deliberately gives that up, and the speed is the direct

payment for it. If you need group membership, collect it separately by grouping elements by their find result.

⊗ Common Misconception

Belief – A union can be undone by reversing the pointer that was changed.

Correction – Standard Union-Find supports no deletion or undo of any kind.

Reason – Path compression has already rewritten parent pointers all over the structure, so the state after a union is not recoverable from the pointer that union changed. Splitting groups requires a different, considerably more complex structure.

26 Best Practices for Union-Find

We recommend keeping the implementation boring, because this is a structure where clever variations mostly cause bugs.

Always implement both optimizations together – they are four extra lines each, and omitting either forfeits the property that makes the structure remarkable.

Prefer *union by size* when you also need group sizes: track the element count per root and attach the smaller group under the larger. This gives the same $O(\alpha(n))$ guarantee as union by rank and hands you a useful extra fact for free.

Maintain a running count of groups, decremented on every successful union. It costs one line and instantly answers “how many separate groups remain?”

Map non-integer keys to indices $0 \dots n - 1$ once, up front, using a hash table – Union-Find wants dense integer indices, and mixing string keys into it complicates the code for no gain.

27 Remember Union-Find Like This

Two matched images carry both optimizations, and are worth fixing in place before moving on.

Memory Anchor

Union-Find is **clubs with presidents**. Each member knows only who they report to. “Same club?” means “same president?” Merging clubs is one handshake between two presidents – the members are never told, and their memberships change anyway.

Memory Anchor

Union by rank is the **small club joining the big club**, so nobody’s chain of command gets longer. Path compression is **promoting everyone you pass**: on your way up to the president, every person you walk past is told “from now on, report straight to the president,” so the next visitor’s walk is shorter.

Key Takeaway

- Union-Find answers “are these two in the same group?” and deliberately nothing else
- Each element stores one parent pointer; a self-pointing element is its group’s representative
- Union by rank prevents tall trees; path compression flattens the paths you walk – use both
- Together they give $O(\alpha(n))$ amortized cost, which is at most about 5 steps for any real input
- union returning false means “already connected” – the exact test Kruskal’s algorithm needs in Chapter 19

28 When Is a Specialized Structure Worth It?

Step back and compare the two halves, because their contrast is the most useful thing to carry out of this chapter.

 Concept Comparison

Compares to: trie versus Union-Find

Table 4. Two specialized structures, one shared design idea.

	Trie	Union-Find
Narrow question it answers	Which keys begin with this?	Are these two in the same group?
What the path means	The path is the key	The path is disposable; only the root matters
What it gives up	Memory	The ability to list a group, or undo
Cost per operation	$O(L)$ in key length	$O(\alpha(n))$ amortized

Table 4 shows two structures with nothing mechanically in common that were nevertheless designed the same way. In each case, someone noticed that a general-purpose structure was doing work the problem did not require, and traded away a capability that was never needed.

That gives a usable test with three questions. **What exactly does my code ask, over and over?** If the answer is a narrow, repeated question rather than “store and retrieve,” a specialized structure may exist for it. **What am I willing to give up?** Every specialized structure trades away generality, and the trade is only good if you truly do not need what it drops. **Is the gain real at my scale?** A trie beating a hash table on prefix queries matters at two million products and does not matter at two hundred.

When the answer to any of these is unclear, the general structure is the right engineering choice. Specialized structures cost reader attention and maintenance effort, and those are real costs even when the complexity analysis looks flattering. Chapter 16 turns this three-question test into a full decision framework covering every structure in the book.

29 Connections

 Connections

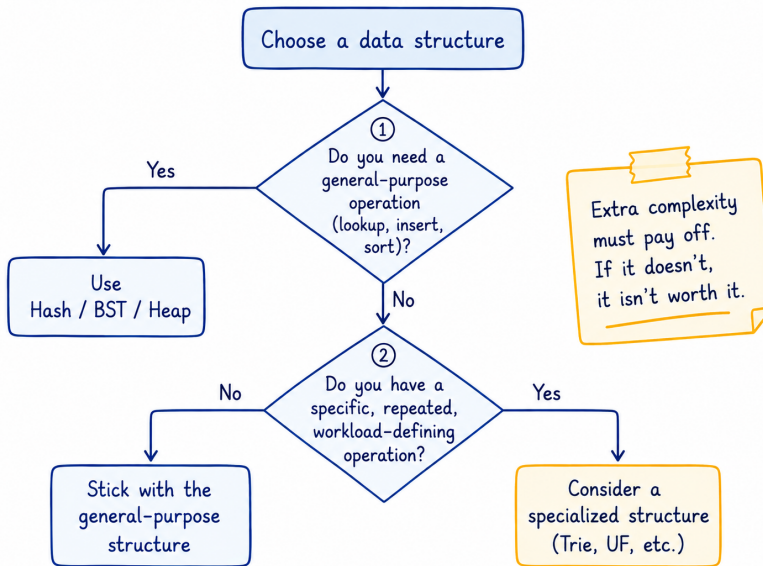


Figure 12. Specialised structures earn their complexity only when the workload actually rewards them.

- **Builds on Chapter 4 (Abstract Data Types):** Union-Find was specified as an ADT – three operations – before any array appeared. The parent array is one implementation of that interface, not the interface itself.
- **Builds on Chapter 13 (Trees, BSTs, and Balance):** both structures are trees, and neither is a search tree. The naive Union-Find chain is the same failure as an unbalanced BST, and union by rank is the same cure: bound the height.
- **Builds toward Chapter 16 (Choosing the Right Data Structure):** the three-question test in this chapter becomes part of the full selection framework.
- **Builds toward Chapter 19 (Minimum Spanning Trees):** Kruskal's algorithm keeps each edge that joins two previously separate groups – the union-returns-false test written here.

30 Key Takeaway

Key Takeaway

- Specialized structures win by exploiting the *shape* of a repeated question, and by giving up a capability the problem never needed
- A trie puts characters on edges and keys on paths; lookup costs $O(L)$ in key length, not $O(n)$ in the number of keys, and the end-of-word marker is what separates a prefix from a stored word
- Union-Find answers “same group?” using one parent pointer per element, with union by rank to prevent tall trees and path compression to flatten the ones you walk
- Both optimizations together give $O(\alpha(n))$ amortized cost – effectively constant, at most about 5 steps for any input a computer can hold
- Reach for a specialized structure only when the narrow query is genuinely repeated, the discarded capability is genuinely unneeded, and the scale is genuinely large

31 Practice

Exercise

Concept Check

Prompt: True or false, with one sentence of justification for each: (a) A trie node stores the character that leads to it. (b) If walk(“pre”) succeeds, then “pre” is a stored key. (c) Trie lookup slows down as more keys are inserted. (d) In Union-Find, $\text{parent}[x] = x$ means x is the representative of its group. (e) $\text{connected}(a, b)$ never modifies the parent array. (f) A Union-Find tree’s shape tells you something meaningful about the elements it contains.

Exercise

Concept Check

Prompt: Fill in the blanks. A trie’s lookup cost is $O(____)$, where that variable means $____$. With union by rank alone, Union-Find’s

worst-case cost per operation is $O(____)$. With both optimizations it is $O(____)$ amortized, which for any realistic input is at most about $____$ steps – and that bound is /is not (circle one) the same thing as $O(1)$.

Exercise

Reflection Question

Prompt: A hash table can find any exact key in $O(1)$ average time, which is faster than a trie's $O(L)$. Explain in your own words why a hash table still cannot answer 'which keys begin with lap?' efficiently, and why the trie's slower exact lookup is not a fair criticism of it.

Exercise

Reflection Question

Prompt: Union-Find cannot list the members of a group. Explain why that restriction is the source of its speed rather than an oversight, and describe how you would obtain group membership if a problem genuinely required it.

Exercise

Reflection Question

Prompt: Union by rank and path compression both make trees shallower. Describe the distinct moment at which each one acts, and construct a short scenario in which one of them does nothing at all while the other does useful work.

Exercise

Coding Exercise

Prompt: Implement `count_words_with_prefix(prefix)` on the trie of `@lst:trie-ops`, returning how many stored keys begin with the given prefix. First write the straightforward version, then describe (in comments) what you would store in each node to answer the same query in $O(L)$ regardless of how many keys match.

 Exercise**Coding Exercise**

Prompt: Implement `delete(word)` for a trie. It must unset the end-of-word marker and remove any nodes that are no longer on the path to any stored key – while never removing a node that another word still needs. Test it on the trie from @fig:15-3-trie-shape by deleting `cart`, then `car`.

 Exercise**Coding Exercise**

Prompt: Convert @lst:union-find from union by rank to union by size, tracking the number of elements in each group. Add a `group_count()` operation returning how many separate groups currently exist, and a `group_size(x)` operation returning the size of `x`'s group.

 Exercise**Coding Exercise**

Prompt: Write `find` recursively, so that path compression happens as the recursion unwinds. Compare it with the two-pass iterative version in @lst:union-find, and state one practical situation in which you would prefer the iterative form.

 Exercise**Debugging Exercise**

Prompt: A student's union is written as: `root_a = find(a); if rank[a] < rank[b]: parent[a] = root_b; else: parent[b] = root_a`. Identify every bug in these three lines, explain what goes wrong for each, and give a short operation sequence on which the buggy version reports two elements as being in different groups when they are actually together.

 Exercise**Debugging Exercise**

Prompt: A trie search is written so that it returns true whenever the walk reaches a node without breaking. Insert cart and care but not car, then find a query for which this code gives the wrong answer. State the one-line fix and explain why the bug is invisible on a dictionary in which no stored word is a prefix of another.

 Exercise**Debugging Exercise**

Prompt: A student adds path compression but forgets union by rank, then reports that their code is ‘fast enough on all my tests.’ Construct an input sequence on which their version is measurably slower than the full one, and explain what property of their tests hid the problem.

 Exercise**Discussion Question**

Prompt: Your team must add autocomplete over 200 million product names, updated hourly, on servers where memory is the scarcest resource. Argue for a trie, then argue against it and for a sorted-array or hash-table alternative. State explicitly what measurement would settle the disagreement.

 Exercise**Discussion Question**

Prompt: Both structures in this chapter gain speed by refusing to answer certain questions. Identify a structure from an earlier chapter that makes a similar trade, and discuss the general risk of adopting a structure whose limitation you have not fully anticipated.

 Exercise**Interview Question**

Prompt: Design a data structure supporting `insert(word)`, `search(word)`, and `starts_with(prefix)`, all in time proportional to the length of the input string. State your time and space complexity, and explain your choice of child container.

Exercise

Interview Question

Prompt: You are given n items and a list of pairs stating that two items are the same. Return the number of distinct items after all merges. State your complexity and justify your choice of structure over repeated searching.

Exercise

Interview Question

Prompt: Given a grid of cells where each cell is either land or water, count the number of separate landmasses, where cells connect horizontally and vertically. Solve it with Union-Find, explaining how you map two-dimensional coordinates to Union-Find indices.

Exercise

Challenge Problem

Optional

Prompt: Sketch a compressed trie, in which any chain of single-child nodes is collapsed into one node storing a whole substring. Describe what insert must do when a new key diverges in the middle of a stored substring, and estimate the memory saved on a dictionary of English words.

Exercise

Challenge Problem

Optional

Prompt: Given a stream of connections and a stream of queries interleaved arbitrarily, design a structure answering ‘how large is the group containing x ?’ in near-constant time while supporting

merges. Then explain precisely why supporting a disconnect operation is far harder than supporting connect.

Further Reading

Sources: Sedgewick & Wayne, Algorithms, 4th ed. – Section 1.5, Case Study: Union-Find; Cormen, Leiserson, Rivest & Stein, Introduction to Algorithms, 3rd ed. – Chapter 21, Data Structures for Disjoint Sets; Skiena, The Algorithm Design Manual, 2nd ed. – Section 3.8, Specialized Data Structures

Readers who want a fourth structure in this family can read *Appendix: Further Topics – Skip Lists*, which presents a randomized alternative to balanced trees. It is optional and non-examinable.

Thank You

If this book has helped you better understand **Algorithms & Data Structures**, then it has already achieved its purpose.

This book is part of the **Kmin Academy of Computer Science**, an initiative dedicated to making high-quality Computer Science education accessible to everyone.

Every suggestion, correction, and word of encouragement helps improve future editions and supports the creation of more free learning resources.

Thank you for being part of this journey.

T-Bao



Support Future Books

If you found this book valuable, consider supporting CSE Team in creating future Computer Science books and educational resources.

Your support enables us to continue researching, writing, reviewing, and publishing high-quality learning materials for the Computer Science community.

Learn deeply. Build meaningfully. Share generously.